T2- Intelligent App Dev (.Net) – Capstone Project Left Shift Program 2026

Workforce Management System (WMS)

1. INTRODUCTION

This case study describes the design and development of a **Workforce Management System (WMS)** for a mid-to-large enterprise aiming to automate and streamline employee-related operations. The system supports core HR and operational workflows including attendance tracking, leave management, employee data management, department structuring, approval workflows, and reporting.

The project integrates a **.NET Core Web API backend**, **Angular frontend**, **SQL Server database**, **JavaScript for UI logic**, and **Azure DevOps CI/CD pipelines** for automated deployment.

---

2. SETUP CHECKLIST

Below are the system and software requirements needed to build and run the Workforce Management System.

Hardware Requirements

- Minimum 8 GB RAM (16 GB recommended)
- Intel i5 processor or above
- Minimum 20 GB free disk space

Software Requirements

- Windows 10/11 or compatible OS
- Visual Studio 2022 (for .NET Core development)
- Visual Studio Code (for Angular development)
- Node.js LTS + Angular CLI
- .NET SDK 8.0
- SQL Server 2019 / Azure SQL
- Git
- Azure DevOps account for CI/CD

Browser Requirements

- Latest versions of Chrome / Edge

---

3. INSTRUCTIONS

1. Follow standard coding guidelines and maintain consistent naming conventions.

2. Maintain a clean **folder structure** for UI, Swagger API, and database projects.
3. Use **Git branching strategies** (main, dev, feature/*)(DB).
4. Perform unit testing before merging code.
5. Maintain an updated **High-Level Design (HLD)** and **Low-Level Design (LLD)** document.
6. Ensure proper **exception handling** and **logging** in backend and frontend.
7. Use **environment-specific configurations** during deployments.
8. Secure sensitive credentials using environment variables or Azure Key Vault.
9. Create Test cases for each module .

---

4. PROBLEM STATEMENT

The current employee management workflow within the organization is fragmented across spreadsheets, emails, and manual processes. This leads to:

- Poor tracking of attendance and working hours
- No centralized task and leave management
- Manual approval workflows
- Difficulty generating HR and compliance reports
- Error-prone employee data handling

A **centralized Workforce Management System** is required to solve these challenges.

---

5. OBJECTIVE

The objective of the Workforce Management System is to provide an integrated platform that:

- Centralizes all employee information
- Ensures accurate attendance and leave tracking
- Automates approval workflows
- Supports department and project management
- Provides real-time dashboards for managers
- Supports analytics for HR compliance and workforce planning

The system must be **scalable, secure, cloud-deployable**, and follow best practices in fullstack architecture.

Complete Table Structures for Workforce Management System

---

1. EMPLOYEE TABLE

Column Name	Data Type	Constraints	Description
EmployeeId	INT	Primary Key, Identity	Unique Employee ID
FirstName	VARCHAR(50)	NOT NULL	Employee First Name
LastName	VARCHAR(50)	NOT NULL	Employee Last Name
Email	VARCHAR(80)	UNIQUE, NOT NULL	Official Email Address
PhoneNumber	VARCHAR(15)	NOT NULL	Employee Contact Number
Gender	CHAR(1)	CHECK (M/F/O)	Gender of Employee
DOB	DATE	NOT NULL	Date of Birth (>= 18 years)
DOJ	DATE	NOT NULL	Date of Joining
DepartmentId	INT	Foreign Key	Allocated Department
RoleId	INT	Foreign Key	Employee Role (Admin/Manager/User)
Status	VARCHAR(20)	DEFAULT 'Active'	Active / Inactive
CreatedOn	DATETIME	DEFAULT GETDATE()	Record Created Date
UpdatedOn	DATETIME	NULL	Record Updated Date

---

2. DEPARTMENT TABLE

Column Name	Data Type	Constraints	Description
DepartmentId	INT	Primary Key, Identity	Unique ID
DepartmentName	VARCHAR(100)	NOT NULL	Department Name
Description	VARCHAR(255)	NULL	Department Description
CreatedOn	DATETIME	DEFAULT GETDATE()	Record Created Date

---

3. ROLE TABLE

Column Name	Data Type	Constraints	Description

RoleId	INT	Primary Key, Identity	Unique Role ID
RoleName	VARCHAR(50)	NOT NULL	Admin / Manager / Employee
Description	VARCHAR(150)	NULL	Role Description

---

4. ATTENDANCE TABLE

Column Name	Data Type	Constraints	Description
AttendanceId	INT	Primary Key, Identity	Unique ID
EmpId	INT	Foreign Key	Employee ID
CheckIn	DATETIME	NOT NULL	Login Time
CheckOut	DATETIME	NULL	Logout Time
TotalHours	FLOAT	Computed	(Checkout – Checkin)
WorkMode	VARCHAR(20)	NULL	WFO / WFH / Hybrid
AttendanceDate	DATE	NOT NULL	Attendance for the day

---

5. LEAVE TABLE

Column Name	Data Type	Constraints	Description
LeaveId	INT	Primary Key, Identity	Unique Leave ID
EmpId	INT	Foreign Key	Employee applying leave
LeaveType	VARCHAR(30)	NOT NULL	Sick / Casual / Earned
Reason	VARCHAR(255)	NULL	Leave Reason
FromDate	DATE	NOT NULL	Start Date
ToDate	DATE	NOT NULL	End Date
Status	VARCHAR(20)	DEFAULT 'Pending'	Pending / Approved / Rejected
AppliedOn	DATETIME	DEFAULT GETDATE()	Leave Applied Date
ApprovedBy	INT	NULL	Manager ID
ApprovedOn	DATETIME	NULL	Date of Approval

---

6. ANNOUNCEMENT / NOTICE BOARD TABLE

Column Name	Data Type	Constraints	Description
AnnouncementId	INT	Primary Key, Identity	Unique ID
Title	VARCHAR(100)	NOT NULL	Announcement Title
Message	VARCHAR(MAX)	NOT NULL	Announcement Body
CreatedBy	INT	Foreign Key	Admin User ID
CreatedOn	DATETIME	DEFAULT GETDATE()	Announcement Date
IsActive	BIT	DEFAULT 1	Whether announcement is visible

---

7. PROJECT TABLE

Column Name	Data Type	Constraints	Description
ProjectId	INT	Primary Key, Identity	Unique ID
ProjectName	VARCHAR(100)	NOT NULL	Name of Project
ClientId	INT	NULL	ForiengKey
StartDate	DATE	NULL	Start Date
EndDate	DATE	NULL	End Date
Status	VARCHAR(20)	DEFAULT 'Active'	Active / Completed

8.Client Table

Column Name	Data Type	Constraints	Description
ClientId	INT	Primary Key, Identity	Unique ID
ClientName	VARCHAR(100)	NOT NULL	Name of Client
ClientAdress	VARCHAR(max)	NULL	Client’s Adress
ClientPhoneNumber	Numeric(10,0)	NULL	Client Phone

ClientLocation	Varchar(20)	NULL	Client Location
Status	Bit	DEFAULT 'Active'	Active / InActive

8. EMPLOYEE–PROJECT ALLOCATION TABLE

Column Name	Data Type	Constraints	Description
AllocationId	INT	Primary Key, Identity	Unique ID
EmpId	INT	Foreign Key	Employee Assigned
ProjectId	INT	Foreign Key	Project Assigned
AssignedOn	DATE	NOT NULL	Assignment Date
CreateDate	DATE	Not Null	Create Date
CreatedBY	Varchar(50)	Not Null	Created By
Status	Bit	Default (Active)	Active/InActive
UpdatedBy	Varchar(50)	Null	Updated By
UpdatedDate	DATE	Null	UpdateDate

9. USER LOGIN TABLE (Authentication)

Column Name	Data Type	Constraints	Description
UserId	INT	Primary Key, Identity	Unique User ID
Username	VARCHAR(50)	UNIQUE	Login Username
PasswordHash	VARCHAR(MAX)	NOT NULL	Hashed Password
RoleId	INT	Foreign Key	Maps to Role Table
LastLogin	DATETIME	NULL	Last Login Timestamp

---

10. AUDIT LOG TABLE

Column Name	Data Type	Description
AuditId	INT	Primary Key

EntityName	**nVarCHAR(Max)**	**Table Updated**
RecordId	**INT**	**Affected Row**
Action	**VARCHAR(20)**	**Insert / Update / Delete**
CreatedBY	**INT**	**User who performed action**
CreatedOn	**DATETIME**	**Timestamp**

6. PROJECT STRUCTURE

A recommended folder/project structure is as follows:

/WMS-Solution

/WMS.API → ASP.NET Core Web API

/WMS.Application → Services, DTOs, Validation

/WMS.Domain → Entities, Interfaces

/WMS.Infrastructure → EF Core, SQL Server

/WMS.Frontend → Angular Client Application

/WMS.Tests → Unit Tests

/WMS.DevOps → Pipeline YAML scripts

Frontend Structure (Angular)

/src

/app

/auth

/employees

/attendance

/leaves

/dashboard

/shared

---

7. EXPECTATION

The final system must:

- Be completely functional end-to-end
- Provide a responsive Angular interface

- Ensure secured API communication using JWT
- Support real-time UI updates using RxJS
- Provide CRUD functionalities for all major modules
- Include proper validation checks
- Deliver production-ready CI/CD pipelines
- Include clear documentation and test evidence
- Use the DataAnotations
- Use Code-First Approach.
- Use Crystal Reports for making any reports.

Grading expectations (if academic):
Code quality
Successful build and deployment
Functional correctness
UI/UX completeness
Documentation completeness

---

8. REQUIREMENT

Functional Requirements

1. **Employee Management**
    - Add, edit, update employee information
    - Search by name, ID, department, role
2. **Attendance Management**
    - Check-in/out
    - View monthly attendance
    - Timesheet reports(crystal Reports)
3. **Leave Management**
    - Apply for leave
    - Cancel leave
    - Manager approval workflow
4. **Department & Project Management**
    - CRUD for departments
    - Assign employees to Project
5. **Dashboard**

- Summary cards
- Attendance chart
- Leave statistics
- Project counts

Non-functional Requirements

- Security: JWT, HTTPS / OAuth /Cors
- Usability: Intuitive UI
- Performance: API < 200ms response
- Scalability: Should support up to 5000 users
- CI/CD: Automated build + deployment

---

9. IMPLEMENTATION

This section outlines how to build each layer.

Backend Implementation (ASP.NET Core Web API)

Steps:

1. Create solution and projects (Domain, Infrastructure, Application, API).
2. Define entities: Employee, Attendance, Leave, Department.
3. Configure DbContext using EF Core.
4. Implement Repository + Service pattern.
5. Add DTOs and AutoMapper for transformations.
6. Implement Proper Validation for business rules.
7. Configure Authentication using JWT.
8. Create controllers for each module.
9. Write unit tests using xUnit.
10. Use DataAnotations.

Frontend Implementation (Angular)

Steps:

1. Create Angular project with routing enabled.
2. Generate feature modules:

    - attendance, employees, leaves, auth, dashboard
3. Use Reactive Forms for validation.
4. Use Angular Material for UI components.
5. Implement state management with RxJS BehaviorSubject.
6. Create Authentication Guard & Interceptor.
7. Consume API using HttpClient services.
8. Build charts using Chart.js.
9. Use Code Optimization.

Database Implementation

Steps:

1. Design schema
2. Create tables using EF Core migrations
3. Apply foreign keys
4. Seed initial data for roles/departments

DevOps Implementation

Steps:

1. Create Azure Repo
2. Configure build pipeline:
    - Restore .NET solutions
    - Build API
    - Run tests
    - Build Angular app
    - Publish artifacts
3. Configure release pipeline:
    - Deploy API to Azure App Service
    - Deploy Angular to Static Web Apps
4. Add test automation stage
5. Add deployment approvals

---

10. SUMMARY OF THE FUNCTIONALITY TO BE BUILT

Below is the final summary of expected deliverables:

Module	**Functionality**
Employee	Add/edit employees, Assign roles, Search,
Attendance	Check-in/out, Timesheet, Monthly view
Leave	Apply, Cancel, Approve/Reject
Projects	Assign, Cancel, Approve/Reject
Auth	Login, Logout, JWT tokens, Role guard
Dashboard	KPIs, charts, counts, alerts
Department	CRUD operations
DevOps	CI/CD pipelines, automated deployment
